Week 42: From Backpropagation to Differential Equations

All derivations, written out

FYS-STK3155/4155 · Monday lecture

Notation (as on the slides). One sample $\boldsymbol{x}$, layers $l = 1, \ldots, L$, $\boldsymbol{a}^0 = \boldsymbol{x}$:

$$\boldsymbol{z}^l = \boldsymbol{W}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \quad \boldsymbol{a}^l = f(\boldsymbol{z}^l),$$

$\boldsymbol{W}^l$ of shape (nodes out, nodes in). The last layer has the output activation f_L. For one node: $z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l$. The error of node j in layer l is

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}.$$

$\circ$ is the element-wise product. Boxes marked **On the board** are the three derivations the slides leave to the board.

Contents

1. Backpropagation: the chain rule, organised . . . 2
2. The general case: a batch of samples . . . 3
3. Quick question 1: why not start at zero? . . . 4
4. Activation functions and their derivatives . . . 4
5. Initialisation: the variance of z . . . 4
6. Output activation and cost: $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$ . . . 5
7. A mismatched pair: sigmoid + squared error . . . 5
8. Quick question 2: which image trains the network? . . . 6
9. Universal approximation: hats from ReLU units . . . 6
10. Exponential decay . . . 8
11. Second derivatives: why ReLU fails . . . 9
12. Boundary conditions built into the trial solution . . . 9
13. Standard methods, for comparison . . . 10
14. Diffusion: a trial solution in two variables . . . 10
15. Soft constraints: conditions as penalties . . . 11
16. Quick question 3: a ReLU PINN . . . 11
17. Inverse problems: an unknown parameter . . . 12

1. Backpropagation: the chain rule, organised

On the board: Chain rule

1.1. Step 1: the output layer

C depends on z_j^L only through $a_j^L = f_L(z_j^L)$:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L}\frac{\mathrm{d}a_j^L}{\mathrm{d}z_j^L} = f_L'(z_j^L)\frac{\partial C}{\partial a_j^L}.$$

1.2. Step 2: one layer back

z_j^l affects C only through a_j^l, and a_j^l goes into *every* node of the next layer,

$$z_k^{l+1} = \sum_j w_{kj}^{l+1} a_j^l + b_k^{l+1} \quad \Rightarrow \quad \frac{\partial z_k^{l+1}}{\partial a_j^l} = w_{kj}^{l+1}.$$

The chain rule sums over all these paths:

$$\delta_j^l = \sum_k \underbrace{\frac{\partial C}{\partial z_k^{l+1}}}_{\delta_k^{l+1}} \underbrace{\frac{\partial z_k^{l+1}}{\partial a_j^l}}_{w_{kj}^{l+1}} \underbrace{\frac{\mathrm{d}a_j^l}{\mathrm{d}z_j^l}}_{f'(z_j^l)} = \left(\sum_k w_{kj}^{l+1}\delta_k^{l+1}\right) f'\left(z_j^l\right).$$

The error of node j comes back along every edge it feeds.

1.3. Step 3: the gradients

$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l$ depends on w_{ji}^l with slope a_i^{l-1} and on b_j^l with slope 1:

$$\frac{\partial C}{\partial w_{ji}^l} = \frac{\partial C}{\partial z_j^l}\frac{\partial z_j^l}{\partial w_{ji}^l} = \delta_j^l a_i^{l-1}, \quad \frac{\partial C}{\partial b_j^l} = \delta_j^l.$$

Weight gradient = error at the end of the edge × signal at its start.

1.4. Step 4: matrix form

In step 2 the sum runs over the *first* index of w_{kj}^{l+1}, which is a multiplication by the transpose. Step 3 for all j, i at once is an outer product:

$$\boldsymbol{\delta}^L = f_L'(\boldsymbol{z}^L) \circ \frac{\partial C}{\partial \boldsymbol{a}^L}, \qquad \boldsymbol{\delta}^l = \left(\left(\boldsymbol{W}^{l+1}\right)^T \boldsymbol{\delta}^{l+1}\right) \circ f'\left(\boldsymbol{z}^l\right),$$
$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \boldsymbol{\delta}^l \left(\boldsymbol{a}^{l-1}\right)^T, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l.$$

Shape check: $\boldsymbol{\delta}^l\left(\boldsymbol{a}^{l-1}\right)^T$ is $(n_l \times 1)(1 \times n_{l-1}) = (n_l, n_{l-1})$, the shape of $\boldsymbol{W}^l$.

Once $\boldsymbol{\delta}^{l+1}$ is known, $\boldsymbol{\delta}^l$ costs one matrix-vector product. That is why we go *backwards*, and why the whole gradient costs about as much as one extra forward pass.

1.5. Step 5 (optional): unroll, and the vanishing gradient

Follow one path from the output back to layer 1. Each layer contributes one factor wf':

$$\delta^1 \propto \delta^L \prod_{l=2}^{L} w^l f'\big(z^{l-1}\big).$$

With the sigmoid, $f' \le \frac{1}{4}$, so the f' factors alone shrink the error by at least $\left(\frac{1}{4}\right)^{L-1}$.

$$\delta_j^L = f_L'\big(z_j^L\big)\frac{\partial C}{\partial a_j^L}, \quad \delta_j^l = \left(\sum_k w_{kj}^{l+1}\delta_k^{l+1}\right) f'\big(z_j^l\big), \quad \frac{\partial C}{\partial w_{ji}^l} = \delta_j^l a_i^{l-1}, \quad \frac{\partial C}{\partial b_j^l} = \delta_j^l$$

2. The general case: a batch of samples

Stack the n samples as **rows**: $\boldsymbol{A}^0 = \boldsymbol{X}$ of shape (n, n_0), and store $\boldsymbol{W}^l$ as (in, out), shape (n_{l-1}, n_l):

$$\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \quad \boldsymbol{A}^l = f\big(\boldsymbol{Z}^l\big),$$

with $\boldsymbol{b}^l$ added to every row. In components, for sample s:

$$Z_{sj}^l = \sum_i A_{si}^{l-1} W_{ij}^l + b_j^l.$$

The cost is the batch mean $C = \frac{1}{n}\sum_s C_s$, and $\Delta_{sj}^l = \partial C / \partial Z_{sj}^l$. The $\frac{1}{n}$ sits in $\partial C / \partial \boldsymbol{A}^L$.

Backwards. Sample s in layer $l+1$ depends only on sample s in layer l: $Z_{sk}^{l+1} = \sum_j A_{sj}^l W_{jk}^{l+1} + b_k^{l+1}$. So

$$\frac{\partial C}{\partial A_{sj}^l} = \sum_k \Delta_{sk}^{l+1} W_{jk}^{l+1} = \left(\boldsymbol{\Delta}^{l+1}\big(\boldsymbol{W}^{l+1}\big)^T\right)_{sj},$$

and multiplying by $f'\left(Z_{sj}^l\right)$ gives $\boldsymbol{\Delta}^l = \left(\boldsymbol{\Delta}^{l+1}\big(\boldsymbol{W}^{l+1}\big)^T\right) \circ f'\big(\boldsymbol{Z}^l\big)$.

Weights. W_{ij}^l appears in Z_{sj}^l for *every* sample s, with slope A_{si}^{l-1}:

$$\frac{\partial C}{\partial W_{ij}^l} = \sum_s \Delta_{sj}^l A_{si}^{l-1} = \left(\big(\boldsymbol{A}^{l-1}\big)^T \boldsymbol{\Delta}^l\right)_{ij}.$$

The matrix product does the sum over the samples.

Biases. b_j^l appears in every row with slope 1: $\partial C / \partial b_j^l = \sum_s \Delta_{sj}^l$, the sum of the rows.

$$\boldsymbol{\Delta}^L = f_L'(\boldsymbol{Z}^L) \circ \frac{\partial C}{\partial \boldsymbol{A}^L}, \quad \boldsymbol{\Delta}^l = \left(\boldsymbol{\Delta}^{l+1}\big(\boldsymbol{W}^{l+1}\big)^T\right) \circ f'\big(\boldsymbol{Z}^l\big),$$

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \big(\boldsymbol{A}^{l-1}\big)^T \boldsymbol{\Delta}^l, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \sum_s \boldsymbol{\Delta}_{s,:}^l$$

Shapes: $\boldsymbol{\Delta}^l$ is (n, n_l); $\big(\boldsymbol{A}^{l-1}\big)^T \boldsymbol{\Delta}^l$ is (n_{l-1}, n_l), the shape of $\boldsymbol{W}^l$. These are the one-sample equations, transposed.

3. Quick question 1: why not start at zero?

All weights and biases are zero.

- **Hidden errors vanish.** $\boldsymbol{W}^{l+1} = \mathbf{0}$ gives $\boldsymbol{\delta}^l = \left(\left(\boldsymbol{W}^{l+1}\right)^T \boldsymbol{\delta}^{l+1}\right) \circ f'\left(\boldsymbol{z}^l\right) = \mathbf{0}$ for every hidden layer, whatever f is. No hidden weight or bias gets a gradient. Only the output layer can move.
- **If $\boldsymbol{f(0) = 0}$** (tanh, ReLU): every hidden $\boldsymbol{a}^l = f(\mathbf{0}) = \mathbf{0}$, so even $\partial C/\partial \boldsymbol{W}^L = \boldsymbol{\delta}^L \left(\boldsymbol{a}^{L-1}\right)^T = \mathbf{0}$. Only the output bias moves: the network learns a constant and never leaves $\boldsymbol{W} = \mathbf{0}$.
- **If $\boldsymbol{f(0) \neq 0}$** (sigmoid): $\boldsymbol{W}^L$ moves first, then layer $L-1$ in the next step, and so on: learning creeps backwards one layer per step, with gradients that are products of tiny weights.
- **Symmetry.** All units in a layer start identical, get identical gradients and stay copies of each other. The same holds for any equal non-zero start. Random weights break the symmetry.

4. Activation functions and their derivatives

Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$\sigma'(z) = \frac{e^{-z}}{\left(1+e^{-z}\right)^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1-\sigma(z)).$$

$\sigma(1-\sigma)$ is a parabola in $\sigma \in (0,1)$ with maximum at $\sigma = \frac{1}{2}$ (that is $z = 0$), so $\sigma' \leq \frac{1}{4}$.

tanh: $\tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, and with the quotient rule

$$\tanh' z = \frac{\left(e^z + e^{-z}\right)^2 - \left(e^z - e^{-z}\right)^2}{\left(e^z + e^{-z}\right)^2} = 1 - \tanh^2 z \leq 1,$$

with equality at $z = 0$. Differentiating once more: $\tanh'' z = -2 \tanh z \cdot \tanh' z$.

ReLU $f(z) = \max(0, z)$: $f'(z) = 1$ for $z > 0$, 0 for $z < 0$, and $f''(z) = 0$ everywhere except at the kink.

Why non-linear? If $f(z) = z$, then $\boldsymbol{a}^2 = \boldsymbol{W}^2(\boldsymbol{W}^1\boldsymbol{x} + \boldsymbol{b}^1) + \boldsymbol{b}^2 = \underbrace{\boldsymbol{W}^2\boldsymbol{W}^1}_{\boldsymbol{W}}\boldsymbol{x} + \underbrace{\boldsymbol{W}^2\boldsymbol{b}^1 + \boldsymbol{b}^2}_{\boldsymbol{b}}$: any number of layers collapses to one linear map.

5. Initialisation: the variance of z

$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{a} + \boldsymbol{b}$ with M inputs. The entries of $\boldsymbol{W}$ and $\boldsymbol{b}$ are independent, with zero mean and variances σ_w^2 and σ_b^2, and independent of $\boldsymbol{a}$. Component by component:

$$\mathbb{E}\left[z_j z_k\right] = \sum_{i,m} \mathbb{E}\left[w_{ji} w_{km}\right] \mathbb{E}[a_i a_m] + \mathbb{E}\left[b_j b_k\right] = \delta_{jk}\left(\sigma_w^2 \sum_i \mathbb{E}[a_i^2] + \sigma_b^2\right),$$

since $\mathbb{E}\left[w_{ji} w_{km}\right] = \sigma_w^2$ only if $j = k$ and $i = m$ (cross terms between $\boldsymbol{W}$ and $\boldsymbol{b}$ vanish because the means are zero). With $\overline{a^2} = \frac{1}{M}\sum_i \mathbb{E}[a_i^2]$:

$$\mathbb{E}[\boldsymbol{z}\boldsymbol{z}^T] = \left(M\sigma_w^2 \overline{a^2} + \sigma_b^2\right)\boldsymbol{I}, \quad \mathrm{Var}(z) = M\sigma_w^2 \overline{a^2} \quad (\boldsymbol{b} = \mathbf{0}).$$

tanh. Near zero $a = \tanh z \approx z$, so $\overline{a^2} \approx \mathrm{Var}(z^{\mathrm{prev}})$ and each layer multiplies the variance by $M\sigma_w^2$. To keep it constant: $\sigma_w^2 = 1/M$.

ReLU. If z^{prev} is symmetric around zero, $a = \max(0, z^{\text{prev}})$ keeps only the positive half:

$$\overline{a^2} = \mathbb{E}[z^2; z > 0] = \frac{1}{2}\mathbb{E}[z^2] = \frac{1}{2}\operatorname{Var}(z^{\text{prev}}) \quad \Rightarrow \quad \operatorname{Var}(z) = \frac{M\sigma_w^2}{2}\operatorname{Var}(z^{\text{prev}}).$$

Constant variance needs $\sigma_w^2 = 2/M$ (He).

ReLU with the tanh rule $\sigma_w^2 = 1/M$: the variance halves in every layer, so the standard deviation shrinks by $\sqrt{2}$. The first layer sees the raw input, so after 10 layers there are 9 such factors: $2^{-9/2} \approx 1/23$. After 50 layers: $2^{-49/2} \approx 4 \times 10^{-8}$.

6. Output activation and cost: $\boldsymbol{\delta^L = a^L - y}$

In every case $\delta^L = f_L'(z^L) \cdot \partial C/\partial a^L$ (one sample).

Regression: linear output, squared error. $a = z$, $C = \frac{1}{2}(a-y)^2$:

$$f_L' = 1, \quad \frac{\partial C}{\partial a} = a - y \quad \Rightarrow \quad \delta^L = a - y.$$

Binary: sigmoid output, cross entropy. $a = \sigma(z)$, $C = -y \ln a - (1-y)\ln(1-a)$:

$$\frac{\partial C}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a} = \frac{-y(1-a) + (1-y)a}{a(1-a)} = \frac{a-y}{a(1-a)},$$

$$\delta^L = \underbrace{a(1-a)}_{\sigma'(z)} \cdot \frac{a-y}{a(1-a)} = a - y.$$

***K* classes: softmax, cross entropy.** $a_k = e^{z_k} / \sum_l e^{z_l}$, $C = -\sum_k y_k \ln a_k$, $\boldsymbol{y}$ one-hot ($\sum_k y_k = 1$). Softmax is not element-wise, so we need the Jacobian:

$$\frac{\partial a_k}{\partial z_j} = \frac{\delta_{kj} e^{z_k} \sum_l e^{z_l} - e^{z_k} e^{z_j}}{\left(\sum_l e^{z_l}\right)^2} = a_k(\delta_{kj} - a_j).$$

Then

$$\delta_j = \sum_k \frac{\partial C}{\partial a_k}\frac{\partial a_k}{\partial z_j} = \sum_k \left(-\frac{y_k}{a_k}\right) a_k(\delta_{kj} - a_j) = -y_j + a_j \sum_k y_k = a_j - y_j.$$

For all three matched pairs f_L' cancels against $\partial C/\partial a$:

$$\boldsymbol{\delta^L = a^L - y} \quad \text{(prediction minus target)}.$$

Each cost is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

7. A mismatched pair: sigmoid + squared error

Sigmoid output, target $y = 1$.

$$\text{cross entropy:} \quad \delta^L_{\text{CE}} = a - 1,$$
$$\text{squared error:} \quad \delta^L_{\text{SE}} = \frac{\partial C}{\partial a}\sigma'(z) = (a-1)\underbrace{a(1-a)}_{\sigma'(z)}.$$

The ratio is $\delta^L_{\text{SE}}/\delta^L_{\text{CE}} = \sigma'(z)$, and $\sigma'(z) \to 0$ as $a \to 0$, which is exactly when the prediction is most wrong. At $z = -5$:

$$a = \sigma(-5) \approx 0.00669, \quad \sigma'(-5) = a(1-a) \approx 0.00665 \approx \frac{1}{150}.$$

δ^L_{SE} is about 150 times smaller than δ^L_{CE}: the more confidently wrong, the *less* it learns.

8. Quick question 2: which image trains the network?

Softmax + cross entropy, so $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$.

- **A, label 0:** $\boldsymbol{a} \approx \boldsymbol{y} = (1, 0, \ldots, 0)$, so $\boldsymbol{\delta}^L \approx \mathbf{0}$. Nothing to learn.
- **B, label 1:** $\boldsymbol{y} = (0, 1, 0, \ldots, 0)$. $\delta_1 = a_1 - 1 = 0.24 - 1 = -0.76$, the only negative component; every other $\delta_k = a_k - 0 = a_k > 0$.

B changes the weights more: raise the score of class 1, lower the rest.

9. Universal approximation: hats from ReLU units

9.1. A hat from three ReLUs (optional board work)

One unit $v\,\text{ReLU}(wx + b)$ is zero, then a straight line: one kink at $x = -b/w$. Add three of them,

$$h(x) = \text{ReLU}(x) - 2\,\text{ReLU}(x-1) + \text{ReLU}(x-2),$$

interval by interval:

$$h(x) = \begin{cases} 0 & x \le 0 \\ x & 0 \le x \le 1 \\ x - 2(x-1) = 2 - x & 1 \le x \le 2 \\ x - 2(x-1) + (x-2) = 0 & x \ge 2 \end{cases}$$

A hat with peak 1 at $x = 1$. Shifted and scaled, with nodes spaced by h:

$$\text{hat}_i(x) = \frac{1}{h}\left[\text{ReLU}(x - x_{i-1}) - 2\,\text{ReLU}(x - x_i) + \text{ReLU}(x - x_{i+1})\right],$$

which is 1 at x_i and 0 at all other nodes. So $G(x) = \sum_i F(x_i)\text{hat}_i(x)$ connects the dots $(x_i, F(x_i))$ with straight lines: a one-hidden-layer ReLU network.

9.2. How accurate? (the $L(b-a)/(2N)$ bound)

Let $|F'| \le L$ on $[a, b]$, split into N intervals of length $h = (b-a)/N$. On $[x_i, x_{i+1}]$ write $x = x_i + th$, $t \in [0, 1]$. Then $G(x) = (1-t)F(x_i) + tF(x_{i+1})$, and

$$F(x) - G(x) = (1-t)(F(x) - F(x_i)) + t(F(x) - F(x_{i+1})).$$

By the mean value theorem $|F(x) - F(x_i)| \le Lth$ and $|F(x) - F(x_{i+1})| \le L(1-t)h$, so

$$|F(x) - G(x)| \le 2t(1-t)Lh \le \frac{Lh}{2} = \frac{L(b-a)}{2N}.$$

Doubling the units halves the error bound.

9.3. Why not a polynomial?

If f is a polynomial of degree p, then each $f\big(\boldsymbol{w}_j^T \boldsymbol{x} + b_j\big)$ is a polynomial of degree at most p in $\boldsymbol{x}$, and so is the sum $G(\boldsymbol{x}) = \sum_j v_j f\big(\boldsymbol{w}_j^T \boldsymbol{x} + b_j\big)$, for *any* N. Polynomials of bounded degree cannot approximate, say, x^{p+1} or e^x arbitrarily well.

10. Exponential decay

On the board: Exponential decay

Problem: $g'(x) + \alpha g(x) = 0$, $g(0) = g_0$, with $\alpha = 2$, $g_0 = 10$. Exact solution $g = g_0 e^{-\alpha x}$.

1. **Trial solution.** We want $g_t(0) = g_0$ *whatever* the network does:
$$g_t(x) = g_0 + xN(x).$$
At $x = 0$ the network term vanishes. (Recipe $g_t = h + BN$ with $h = g_0$, $B = x$.)
2. **Derivative** (product rule):
$$g_t'(x) = N(x) + xN'(x).$$
3. **Residual:**
$$r(x) = g_t' + \alpha g_t = N + xN' + \alpha(g_0 + xN).$$
$r = 0$ for the exact solution.
4. **Cost:** mean squared residual at collocation points $x_i \in [0, 1]$ that we choose:
$$C = \frac{1}{n}\sum_{i=1}^{n} r(x_i)^2.$$
Training changes the weights in N to make C small.
5. **What is new:** $N'(x)$ is a derivative with respect to the *input*, not the weights. Backpropagation for the weights is unchanged; it is regression with target $r = 0$.

$$g_t = g_0 + xN(x), \quad r = \underbrace{N + xN'}_{g_t'} + \alpha(g_0 + xN)$$

Two hidden layers of 40 tanh units, 50 collocation points: max error 3×10^{-3}.

10.1. Same recipe: logistic growth

$g' - 2g(1 - g) = 0$, $g(0) = 1.2$. Trial solution $g_t = 1.2 + tN(t)$,

$$r = N + tN' - 2g_t(1 - g_t).$$

Exact solution (separable equation):

$$g(t) = \frac{1}{1 + (1/g_0 - 1)e^{-2t}} = \frac{1}{1 - e^{-2t}/6}.$$

The residual is non-linear in N; nothing else changes.

11. Second derivatives: why ReLU fails

On the board: Second derivatives

Poisson $g'' + f = 0$, diffusion $\partial_t g - \partial_x^2 g = 0$ and wave $\partial_t^2 g - c^2 \partial_x^2 g = 0$ all contain a second derivative, so we need N''. One hidden layer:

1. $$N(x) = \sum_j v_j f(w_j x + b_j) + c$$

2. Chain rule, w_j comes out:

$$N'(x) = \sum_j v_j w_j f'(w_j x + b_j)$$

3. Once more:

$$N''(x) = \sum_j v_j w_j^2 f''(w_j x + b_j)$$

4. ReLU: $f'' = 0$ everywhere (except at the kink), so $N'' = 0$ for **any** weights.

This is not about training: a ReLU network is piecewise linear, so it has no curvature at all. More layers do not help: compositions of piecewise-linear functions are piecewise linear.

$$N''(x) = \sum_j v_j w_j^2 f''(w_j x + b_j)$$

ReLU: $f'' = 0$, so $N'' = 0$; it cannot solve $g'' + f = 0$.
tanh: smooth, and both derivatives come from f itself: $f' = 1 - f^2$, $f'' = -2ff'$.

$\max|N''|$ of a randomly initialised network: tanh 2.9, GELU 1.2, sigmoid 0.06, ReLU 0.

12. Boundary conditions built into the trial solution

Recipe: $g_t = h(x) + B(x)N(x)$, with h satisfying the conditions and $B = 0$ wherever a condition is imposed.

$g(0) = a, g(1) = b$: $g_t = a + (b - a)x + x(1 - x)N$.

$$g_t(0) = a + 0 + 0 = a, \quad g_t(1) = a + (b - a) + 0 = b.$$

$g(0) = g_0, g'(0) = v_0$: $g_t = g_0 + v_0 x + x^2 N$.

$$g_t(0) = g_0, \quad g_t' = v_0 + 2xN + x^2 N' \Rightarrow g_t'(0) = v_0.$$

$g(0) = a, g'(1) = b$ (needs a trick): $g_t = a + bx + x(N(x) - N(1) - N'(1))$.

$g_t(0) = a, \quad g_t' = b + (N(x) - N(1) - N'(1)) + xN'(x) \Rightarrow g_t'(1) = b + 0 - N'(1) + N'(1) = b.$

12.1. Poisson residual

$g'' + (3x + x^2)e^x = 0$, $g(0) = g(1) = 0$, trial solution $g_t = x(1 - x)N$:

$$g_t' = (1-2x)N + x(1-x)N',$$
$$g_t'' = -2N + 2(1-2x)N' + x(1-x)N''.$$

The residual $r = g_t'' + (3x+x^2)e^x$ contains N'', so the previous section applies. Exact solution $g = x(1-x)e^x$; check:

$$g' = (1-x-x^2)e^x, \quad g'' = (-1-2x)e^x + (1-x-x^2)e^x = -(3x+x^2)e^x.$$

13. Standard methods, for comparison

Grid $t_n = n\Delta t$, $g_n \approx g(t_n)$, equation $g' = F(t, g)$.

Forward Euler: follow the slope for one step,

$$g_{n+1} = g_n + \Delta t F(t_n, g_n). \quad \text{Error} \propto \Delta t.$$

RK4: four slope evaluations per step,

$$k_1 = F(t_n, g_n), \qquad k_2 = F(t_n + \Delta t/2, g_n + \Delta t k_1/2),$$
$$k_3 = F(t_n + \Delta t/2, g_n + \Delta t k_2/2), k_4 = F(t_n + \Delta t, g_n + \Delta t k_3),$$

$$g_{n+1} = g_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad \text{Error} \propto \Delta t^4.$$

Finite differences for $g'' + f = 0$: Taylor expand around x_i,

$$g_{i\pm1} = g_i \pm \Delta x g_i' + \frac{\Delta x^2}{2} g_i'' \pm \frac{\Delta x^3}{6} g_i''' + O(\Delta x^4) \Rightarrow \frac{g_{i+1} - 2g_i + g_{i-1}}{\Delta x^2} = g_i'' + O(\Delta x^2).$$

With $g_0 = g_N = 0$ this is a tridiagonal linear system $-g_{i-1} + 2g_i - g_{i+1} = \Delta x^2 f(x_i)$. Error $\propto \Delta x^2$.

	method	max error	time
logistic	Euler, $\Delta t = 0.01$	9×10^{-4}	0.04 ms
	RK4, $\Delta t = 0.01$	2×10^{-10}	0.14 ms
	network	1×10^{-4}	0.5 s
Poisson	FD, $\Delta x = 0.01$	2×10^{-5}	0.12 ms
	network	1×10^{-5}	0.6 s

14. Diffusion: a trial solution in two variables

$$\partial_t g - \partial_x^2 g = 0, \quad g(0,t) = g(1,t) = 0, \quad g(x,0) = \sin \pi x.$$

Exact solution: $g = e^{-\pi^2 t} \sin \pi x$. Check: $\partial_t g = -\pi^2 g$ and $\partial_x^2 g = -\pi^2 g$, so the residual is zero; $g(x,0) = \sin \pi x$ and $\sin 0 = \sin \pi = 0$.

Trial solution:

$$g_t = (1-t)\sin \pi x + x(1-x)tN(x,t).$$

- $t = 0$: $g_t = \sin \pi x$ (the N term has the factor t).

- $x = 0$ or $x = 1$: $\sin \pi x = 0$ and $x(1-x) = 0$, so $g_t = 0$.

Derivatives for the residual:

$$\partial_t g_t = -\sin \pi x + x(1-x)(N + t\partial_t N),$$
$$\partial_x^2 g_t = -(1-t)\pi^2 \sin \pi x + t\big[-2N + 2(1-2x)\partial_x N + x(1-x)\partial_x^2 N\big],$$

and $r = \partial_t g_t - \partial_x^2 g_t$ at collocation points (x_i, t_i). Again $\partial_x^2 N$ appears: no ReLU.

15. Soft constraints: conditions as penalties

Let the network itself be the solution, $u(x,t) \approx N(x,t)$, and put each condition into the cost:

$$L = L_{\text{PDE}} + \lambda(L_{\text{IC}} + L_{\text{BC}}),$$

$$L_{\text{PDE}} = \frac{1}{n}\sum_i \left(\partial_t N - \partial_x^2 N\right)_i^2 \qquad \text{inside the domain,}$$
$$L_{\text{IC}} = \frac{1}{n}\sum_i \left(N(x_i, 0) - \sin \pi x_i\right)^2 \quad \text{at } t = 0,$$
$$L_{\text{BC}} = \frac{1}{n}\sum_i \Big(N(0,t_i)^2 + N(1,t_i)^2\Big) \text{ at } x = 0, 1.$$

Each term that the trial solution enforced exactly is now a penalty. A new condition is a new term; λ weighs the terms, like the penalty in Ridge.

	RMSE	error at $t = 0$
hard (g_t)	1.8×10^{-3}	0 exactly
soft, $\lambda = 10$	6.5×10^{-3}	6×10^{-3}

16. Quick question 3: a ReLU PINN

Same soft cost, but $u = N(x,t)$ is a ReLU network.

1. ReLU gives $\partial_x^2 N = 0$ at every sample, so
$$L_{\text{PDE}} = \frac{1}{n}\sum_i (\partial_t N)_i^2.$$
2. The cheapest way to make all three terms small: $\partial_t N = 0$ (no time dependence), $N(x,0) \approx \sin \pi x$ (IC), and $N(0,t) = N(1,t) = 0$ (BC, already true for $\sin \pi x$). So
$$u \approx \sin \pi x \quad \text{for all } t.$$
(Piecewise linear in x, but close to $\sin \pi x$.)
3. The true solution decays as $e^{-\pi^2 t}\sin \pi x$, so the network never diffuses: small cost (6×10^{-4}), wrong answer (RMSE 0.65 vs. 1.8×10^{-3}).

A small residual is not a solution. Use smooth activations, and check against something you know.

17. Inverse problems: an unknown parameter

$\partial_t u - D\partial_x^2 u = 0$ with D unknown, no initial or boundary conditions, only 40 noisy measurements y_i. Treat D as one more trainable parameter:

$$L(N, D) = \frac{1}{n}\sum_j \underbrace{\left(\partial_t N - D\partial_x^2 N\right)_j}_{r_j}^2 + \lambda \frac{1}{40}\sum_{i=1}^{40} \left(N(x_i, t_i) - y_i\right)^2.$$

Its gradient with respect to D is one line:

$$\frac{\partial L}{\partial D} = -\frac{2}{n}\sum_j r_j \left(\partial_x^2 N\right)_j,$$

and gradient descent updates D together with the weights. No trial solution is possible: the conditions are unknown, so soft constraints are the only option.

True $D = 0.5$; three starting guesses all give $D \approx 0.51$. With five times more noise, $D = 0.69$: a recovered parameter needs an error bar (for example from the bootstrap).